Programmierung 1

im Wintersemester 2026/27

Praktikumsaufgabe 1

Einstieg in Java (Kap. 1)

Empfohlene Fertigstellungstermine:
3. November 2026 für DFIW und 4. November 2026 für PIB

Mit diesen ersten Aufgaben steigen Sie tiefer in die Welt der Java-Programmierung ein: In Aufgabe 1 experimentieren Sie mit einzelnen Java-Anweisungen. In Aufgabe 2 übertragen Sie diese anschließend in ein vollständiges Java-Programm und entwickeln es weiter. In Aufgabe 3 können Sie dann einen 15-minütigen Crashkurs zu den wichtigsten Grundlagen von Java absolvieren. In Aufgabe 4 können Sie schließlich Ihre individuelle Entdeckungsreise starten.

Lassen Sie sich nicht von der Länge der Aufgaben abschrecken. Die Aufgaben sind so konzipiert, dass Sie sie in kleinen Schritten lösen können, insbesondere als Einsteiger:in ohne Vorkenntnisse in der Programmierung.[1] Um Ihnen den Einstieg zu erleichtern, sind die einzelnen Schritte ausführlich beschrieben. Dies gilt insbesondere für Aufgabe 1. In Aufgabe 2 festigen Sie außerdem den Umgang mit den Werkzeugen, die Sie in dieser Veranstaltung verwenden. Die Aufgaben 3 und 4 entsprechen eher dem Detaillierungsgrad, den Sie in den folgenden Praktikumsaufgaben erwarten können.

Als grobe Orientierung können Sie für die beiden Kernaufgaben 1 und 2 zusammen etwa 90 bis 120 Minuten einplanen, ohne die Bonusaufgaben 3 und 4 zur Vertiefungen. Dabei gehen wir von einem vorbereiteten Buchkapitel und einer funktionierenden Arbeitsumgebung aus. Das ist

[1] Gegen Ende der Vorlesungszeit und damit in Richtung Klausur werden die Aufgabenbeschreibungen deutlich weniger detailliert ausfallen. Der Grund dafür ist, dass die Aufgabenstellungen in der Klausur wesentlich kürzer sein werden als die in dieser und den folgenden Praktikumsaufgaben. So können Sie sich spätestens dann auf die Art der Aufgabenstellung vorbereiten, die Sie in der Klausur erwartet.

eine erste Schätzung, keine Sollzeit: Wie viel Zeit Sie zum Lesen, Ausprobieren, Besprechen und Nachdenken benötigen, darf unterschiedlich sein.

Sie haben also bereits Kapitel 1 des Buchs vorbereitet und eine funktionierende Arbeitsumgebung mit einem JDK, einem Texteditor und einem Terminal eingerichtet. Dort haben Sie bereits ein Hallo-Welt-Programm geschrieben und ausgeführt. Daran knüpfen wir jetzt an. Falls Ihre Umgebung noch nicht funktioniert, nutzen Sie die Hinweise zu den in der Veranstaltung genutzten Programmierwerkzeugen und fragen Sie Ihre Lernbegleiter:innen.

Arbeiten Sie in Ihrem Coding-Team zu zweit an einem Computer und nutzen Sie Pair Programming. Wechseln Sie die Rollen spätestens beim Übergang zur nächsten Aufgabe. Nehmen Sie sich vor dem Ausprobieren kurz Zeit, jeweils eine eigene Vermutung zu bilden, und vergleichen Sie Ihre Ideen. So können beide mitdenken, ausprobieren und erklären.

Um Ihnen das Verstehen und Lernen zu erleichtern, finden Sie bei jeder Aufgabe die Lernziele, die Sie mit dieser Aufgabe erreichen sollen. Diese Lernziele helfen Ihnen, sich auf das Wesentliche zu fokussieren und so effektiv wie möglich zu lernen. Warum? Weil die Lernziele die Frage beantworten: "*Wozu* soll ich das tun und lernen?" Und da Sie in dieser Lernveranstaltung eigenverantwortlich lernen und machen, können Sie jedes Lernziel abhaken, sobald Sie denken, dass Sie es erreicht haben.

Tipp: Um effektiv zu lernen, machen Sie sich immer wieder klar, warum und wofür Sie etwas lernen! Das hilft Ihnen, sich zu motivieren und Ihren Fokus und Ihre Aufmerksamkeit aufrechtzuerhalten.

Heads up! Zu diesem ersten Übungsblatt gibt es kein reguläres Review mit Tutor:in und keine einzureichende Reflexion. Der Wissenstest zu Kapitel 1 gehört weiterhin zur Prüfungsvorleistung. Die Aufgaben sollen Ihnen den Einstieg in Java erleichtern und Sie mit dem Arbeitsablauf vertraut machen, den Sie in dieser Veranstaltung immer wieder verwenden. Nutzen Sie die Präsenztermine des Praktikums am 27. Oktober (DFIW) bzw. am 28. Oktober (PIB), um Ihre Fragen zu klären und sich bei der Lösung unterstützen zu lassen.

Wir als Ihre Lernbegleiter:innen unterstützen Sie gerne bei der Lösung der Aufgaben.

1 Ihr erster Java-Code

Lernziele

- ☐ Sie ändern und erweitern bestehenden Java-Code, um Ihren ersten eigenen Java-Code zu entwickeln.
- ☐ Sie verwenden die JShell, um Java-Code schnell und einfach auszuprobieren und auszuführen.
- ☐ Sie führen eine kurze Selbstreflexion durch, um Ihre Metakognition[a] zu verbessern und eine für Sie passende Lernstrategie zu entwickeln.

[a] Metakognition ist das "Denken über das Denken" und spielt eine wichtige Rolle für erfolgreiches Lernen. Es geht darum, das eigene Denken und Lernen bewusst wahrzunehmen, zu steuern und zu reflektieren. Sie lernen nicht nur, "wie" sie etwas tun, sondern auch "warum" und "wie gut". Das kann Ihnen helfen, Ihr weiteres Lernen gezielt zu gestalten.

Indem Sie ein paar Zeilen *Java-Code* schreiben und ausführen, machen Sie Ihre ersten Schritte als Java-Programmierer:in. Schnappen Sie sich Ihren Laptop, eine Tasse Kaffee oder Tee – und legen Sie los! Sie haben keinen eigenen Computer? Kein Problem, Sie können einen der Arbeitsplätze im STL benutzen. Nur Kaffee oder Tee darf man dort leider nicht trinken. Aber so schlimm ist das ja auch wieder nicht…

1.1 Die JShell starten

Das Buch betrachtet in Kapitel 1 bereits vollständige Java-Programme. Wir untersuchen zunächst einzelne Anweisungen und setzen sie in Aufgabe 2 zu einem vollständigen Programm zusammen. Mit der JShell können Sie solche Anweisungen direkt ausprobieren, ohne selbst ein Programmgerüst zu schreiben. Sie gehört seit Java 9 zum JDK. Für die Beispiele dieser Aufgabe genügt ein JDK ab Version 9; verwenden Sie Ihre bereits eingerichtete Umgebung.

Öffnen Sie Ihr Terminal. Geben Sie dort `jshell` ein und drücken Sie die Eingabetaste. Nun erscheint die Eingabeaufforderung `jshell>`. Sie zeigt an, dass Sie sich in der JShell befinden. Tippen Sie diese Eingabeaufforderung nicht mit!

Wichtig: Die JShell versteht Java-Code. Die Betriebssystem-Shell, aus der Sie sie gestartet haben, versteht dagegen Befehle zum Starten von Programmen wie `jshell`, `javac` und `java`. Beides verwenden Sie im Terminal, aber zu unterschiedlichen Zeitpunkten.

1.2 Den ersten Java-Code ausführen

Geben Sie die folgenden Zeilen nacheinander ein und bestätigen Sie jede mit der Eingabetaste:

```
String text = "Welt";
System.out.println("Hallo " + text + "!");
```

Nach der ersten Zeile meldet die JShell den Wert der Variable, beispielsweise `text ==> "Welt"`. Das ist eine Rückmeldung der JShell. Die zweite Zeile erzeugt die eigentliche Begrüßung: `Hallo Welt!`. Schon haben Sie Ihre ersten Anweisungen in der JShell ausgeführt!

1.3 Die ersten Änderungen an Java-Code machen

Was passiert wohl, wenn Sie jetzt die folgende Anweisung eingeben?

```
text = "Java";
```

Überlegen Sie zunächst beide für sich, ob dadurch bereits eine neue Begrüßung ausgegeben wird. Vergleichen Sie Ihre Vermutungen und probieren Sie es aus. Führen Sie anschließend die Ausgabeanweisung erneut aus:

```
System.out.println("Hallo " + text + "!");
```

Die Zuweisung verändert den gespeicherten Wert. Erst die Ausgabeanweisung gibt die neue Begrüßung aus. Die JShell merkt sich Ihre Variable während der Sitzung. Sie müssen sie deshalb nicht jedes Mal neu anlegen.

Wie könnten Sie nun `Hello, world!` ausgeben? Probieren Sie verschiedene Texte aus! Wenn Sie mögen, ergänzen Sie ein Emoji Ihrer Wahl.[2] Bleiben Sie neugierig: Sie sehen unmittelbar, was Ihre Änderungen bewirken.

[2] Wird Ihr Emoji nicht richtig angezeigt? Das kann an der Terminaldarstellung oder Zeichenkodierung liegen. Probieren Sie zunächst einen Text ohne Emoji aus. Auch bei ungewöhnlich dargestellten Umlauten helfen Ihnen Ihre Lernbegleiter:innen.

Tipp: Mit der Pfeiltaste nach oben können Sie frühere Eingaben wieder aufrufen, bearbeiten und mit der Eingabetaste erneut ausführen.

1.4 Die ersten Lernerfolge feiern

Haben Sie den Java-Code entsprechend angepasst und ausgeführt? Erscheint die gewünschte Ausgabe im Terminal? Herzlichen Glückwunsch! Sie haben Ihren ersten Java-Code geschrieben und ausgeführt. Und das in kürzester Zeit. So macht Lernen Spaß! Und weil effektives Lernen Erfolge feiert: Klopfen Sie sich und Ihrer Coding-Partner:in auf die Schulter! Zelebrieren Sie kleine und große Lernerfolge! Und das Beste: Das ist noch nicht alles. Sie werden noch viele weitere Gelegenheiten haben, Lernerfolge zu feiern. Also: Weitermachen!

1.5 Die erste professionelle Verbesserung

Unser Variablenname `text` ist ziemlich allgemein. Was enthält die Variable eigentlich? Wählen Sie einen Namen, der dies genauer beschreibt, etwa `name` oder `recipient`. Besprechen Sie, welcher Name Ihnen am besten gefällt und warum. Auch verständliche Namen gehören zum Programmieren: Andere Menschen sollen Ihren Code lesen und verstehen können.

Für diesen Versuch beginnen Sie mit einer leeren JShell-Sitzung. Geben Sie dazu `/reset` ein. Dadurch werden Ihre bisherigen Variablen und Anweisungen verworfen. Geben Sie anschließend die beiden Ausgangszeilen erneut ein, diesmal mit Ihrem gewählten Variablennamen.

Tipp: Ändern Sie den Namen in beiden Zeilen. Ohne den Neustart könnte die alte Variable noch vorhanden sein und ein vergessenes Umbenennen verdecken. Wenn jetzt eine Fehlermeldung erscheint, vergleichen Sie die Namen genau. Hurra, Sie dürfen sich irren und daraus lernen!

1.6 Die nächsten Schritte machen

Warum probieren Sie nicht weiter aus? Ändern Sie den Code so ab, dass der Text `Salut le monde !` ausgegeben wird. Achten Sie auch auf das Leerzeichen vor dem Ausrufezeichen. Dabei sollte der Textteil `le monde` möglichst *variabel* sein. Huh, wie könnte *das* gehen? Überlegen

und diskutieren. Dann ausprobieren! So können Sie die schnelle und einfache Ausführung von Java-Code weiter nutzen, um spielerisch zu lernen.

Tipp: Das +-Zeichen ist der *Operator* für die *Verkettung* von *Strings* in Java. Sie können also zwei Zeichenketten mit + zu einer neuen Zeichenkette verbinden.[3]

Wenn Sie den Code nicht so ändern können, dass der Text `Salut le monde !` auf diese Weise ausgegeben wird, ist das kein Problem. Vergleichen Sie Ihre Anweisung mit dem Ausgangsbeispiel oder fragen Sie Ihre Coding-Partner:in und Ihre Lernbegleiter:innen. Probieren Sie anschließend weiter!

1.7 Reflexion Ihrer Lernerfahrung

Nachdem Sie Ihre ersten Schritte in der Java-Programmierung gemacht haben, nehmen Sie sich bitte ein paar Minuten Zeit, um über Ihren Lernprozess nachzudenken. Beantworten Sie die folgenden Fragen schriftlich oder im Gespräch:

1. Was habe ich gelernt?

Beschreiben Sie in einigen Sätzen, welche neuen Konzepte oder Techniken Sie gelernt haben. Was hat Ihnen geholfen, diese Konzepte zu verstehen?

2. Welche Schwierigkeiten gab es?

Überlegen Sie, ob Sie bei bestimmten Schritten (z. B. beim Ändern oder Ausführen des Codes) auf Schwierigkeiten gestoßen sind. Wo genau sind Sie hängengeblieben? Was hat Sie davon abgehalten, weiterzumachen?

3. Wie habe ich diese Schwierigkeiten gelöst? Wenn nicht, wie könnte ich sie beim nächsten Mal lösen?

Überlegen Sie, wie Sie das Problem gelöst haben (z. B. durch Ausprobieren, Nachschlagen, mit Hilfe einer Kommiliton:in oder durch Fragen im Forum). Wenn Sie keine Lösung gefunden haben, was könnten Sie beim nächsten Mal anders machen?

[3] Weitere Fachbegriffe, die Ihnen bald leicht über die Lippen gehen.

1.7.1 Beispielhafte Reflexion

Eine mögliche Reflexion könnte beispielsweise wie folgt aussehen:

1. Ich habe gelernt, dass sich der von einem Programm ausgegebene Text ganz einfach in den Anführungszeichen ändern lässt. Außerdem habe ich verstanden, dass eine Variable wie ein Behälter für Informationen ist und dass sie an allen Stellen, an denen sie verwendet wird, exakt gleich heißen muss.

2. Nach dem Neustart der JShell hatte ich bei der Umbenennung der Variable `text` zunächst vergessen, sie auch in der zweiten Zeile (`println`) zu ändern. Die Ausgabeanweisung ließ sich dann nicht ausführen und ich wusste kurz nicht, warum.

3. Ich habe meinen Code genau mit dem Originalcode aus der Anleitung verglichen und so den Fehler gefunden. Nächstes Mal achte ich darauf, eine Änderung an allen betroffenen Stellen durchzuführen.

1.8 Zurück zur Betriebssystem-Shell

Geben Sie `/exit` ein, um die JShell zu verlassen. Nun können Sie im Terminal wieder Programme wie den Java-Compiler starten. Genau damit geht es in der nächsten Aufgabe weiter.

2 Vom Experiment zum Java-Programm

Lernziele

- ☐ Sie verändern und erweitern Code in einem vollständigen Java-Programm.
- ☐ Sie speichern Quelltext, übersetzen ihn mit `javac` und führen das Programm mit `java` aus.
- ☐ Sie öffnen gespeicherten Quelltext wieder und bearbeiten ihn weiter.

Beim Einrichten Ihrer Arbeitsumgebung haben Sie bereits ein Hallo-Welt-Programm lokal ausgeführt. Nun übertragen Sie Ihre Experimente aus Aufgabe 1 in ein solches Programm. Dabei festigen Sie den Arbeitsablauf, den Sie in den kommenden Wochen immer wieder verwenden. Wechseln Sie vorher die Rollen in Ihrem Coding-Team!

2.1 Das Programmgerüst kennenlernen

In der JShell konnten Sie einzelne Anweisungen direkt ausführen. Für unser vollständiges Programm verwenden wir eine Klasse namens `Main` mit einer Methode namens `main`. Diese Methode ist der Einstiegspunkt unseres Programms:

```
1  public class Main {
2
3    public static void main(String[] args) {
4      // Hier stehen die Anweisungen, die beim Programmstart ausgeführt
       ↪ werden.
5      System.out.println("Hallo Welt!");
6    }
7
8  }
```

Machen Sie sich nicht die Mühe, schon jedes Wort dieses Gerüsts im Detail zu erklären. Schauen Sie dazu auch in Kapitel 1 des Buches. Wichtig ist jetzt, *wo* Ihre eigenen Anweisungen stehen: innerhalb der geschweiften Klammern der Methode `main`. Dort sehen Sie bereits die bekannte Ausgabeanweisung.

2.2 Die Datei vorbereiten

Legen Sie einen eigenen Ordner für diese Aufgabe an, beispielsweise `pr1-kapitel01`. Öffnen Sie Ihren Texteditor, schreiben Sie das obige Programm und speichern Sie es in diesem Ordner als `Main.java`.

Achten Sie auf die genaue Schreibweise: Die öffentliche Klasse heißt `Main`, deshalb heißt die Datei `Main.java`. Speichern Sie reinen Text, kein formatiertes Textdokument. Prüfen Sie auch, dass Ihr Editor nicht unbemerkt eine Endung ergänzt hat: `Main.java.txt` wäre der falsche Dateiname.

2.3 Übersetzen und ausführen

Falls Sie noch die Eingabeaufforderung `jshell>` sehen, verlassen Sie die JShell mit `/exit`. Die folgenden Befehle geben Sie in der *Betriebssystem-Shell* ein.

Wechseln Sie im Terminal in Ihren Aufgabenordner. Dazu dient `cd`, gefolgt vom Pfad des Ordners. Setzen Sie einen Pfad mit Leerzeichen in gerade Anführungszeichen. Den vollständigen Pfad können Sie in Ihrer Dateiverwaltung ermitteln. Beispielsweise sieht ein Wechsel unter Windows so aus:

```
cd "C:\Users\IhrName\pr1-kapitel01"
```

Unter macOS könnte der vollständige Pfad `/Users/IhrName/pr1-kapitel01` lauten, unter Linux `/home/IhrName/pr1-kapitel01`. Verwenden Sie jeweils den tatsächlichen Speicherort Ihres Ordners. Mit `dir` unter Windows bzw. `ls` unter macOS oder Linux können Sie den Ordnerinhalt anzeigen. Sehen Sie dort Ihre Datei `Main.java`?

Übersetzen Sie nun den gespeicherten Quelltext:

```
javac Main.java
```

Wenn alles klappt, erscheint normalerweise keine Erfolgsmeldung. Dafür finden Sie nun im Ordner die Datei `Main.class`. Sie enthält den vom Compiler erzeugten Bytecode. Erscheint eine Fehlermeldung, lesen Sie diese, korrigieren Sie Ihren Quelltext, speichern Sie ihn und versuchen Sie es erneut.

Starten Sie nach erfolgreicher Übersetzung das Programm:

```
java Main
```

Hier verwenden Sie den Klassennamen `Main`, ohne `.java` oder `.class`. Sehen Sie die Begrüßung? Geschafft! Sie haben den Weg vom Quelltext über die Übersetzung bis zur Ausführung selbst durchlaufen.

2.4 Die eigenen Schritte machen

Jetzt begleiten wir Sie weniger eng. Übertragen Sie die Ideen aus der JShell in Ihr Programm:

Ändern: Lassen Sie den ausgegebenen Text von einer Variable abhängen. Wählen Sie einen verständlichen Namen.

Erweitern: Speichern Sie `le monde` in einer Variable. Lassen Sie Ihr Programm damit die Begrüßung `Salut le monde !` ausgeben.

Ausprobieren: Erfinden Sie weitere Begrüßungen. Vermuten Sie zunächst, was Ihr Code ausgeben wird, und überprüfen Sie Ihre Vermutung. Wenn Sie weiterforschen möchten, schreiben Sie zwei Ausgabeanweisungen hintereinander und ersetzen Sie bei der ersten `println` durch `print`. Was ändert sich? Vergleichen Sie mit der Erklärung auf Seite 15 des Buches.

Tipp: Die Java-Anweisungen aus Aufgabe 1 passen unverändert in die Methode `main`. Die Eingabeaufforderung `jshell>`, Rückmeldungen der JShell und Befehle wie `/reset` gehören dagegen nicht in Ihre Java-Datei. Wenn Sie eine Variable verwenden, muss sie im Programm zuvor deklariert sein.

Nach jeder Änderung lautet der Ablauf: *im Editor speichern, mit `javac` übersetzen und mit `java` ausführen*.

2.5 Ein kleines Experiment: Welche Fassung läuft?

Beginnen Sie mit einem erfolgreich übersetzten und ausgeführten Programm. Ändern Sie nun den Begrüßungstext im Editor und speichern Sie die Datei. Was wird passieren, wenn Sie jetzt nur `java Main` ausführen? Überlegen Sie beide zunächst für sich und probieren Sie es dann aus.

Übersetzen Sie anschließend mit `javac Main.java` neu und starten Sie noch einmal mit `java Main`. Erklären Sie sich gegenseitig den Unterschied. Welche Rolle spielen dabei `Main.java` und `Main.class`?

2.6 Fehler machen und daraus lernen

In der Buchübung „SEIEN Sie der Compiler“ auf Seite 21 prüfen Sie selbst, welcher Code kompiliert werden kann. Jetzt untersuchen Sie, wie `javac` auf gezielt eingebaute Fehler reagiert.

Entfernen Sie probeweise das Semikolon am Ende der Ausgabeanweisung. Speichern Sie und versuchen Sie zu kompilieren. Welche Stelle nennt die Fehlermeldung? Stellen Sie das Semikolon wieder her und übersetzen Sie erneut.

Probieren Sie anschließend einen falsch geschriebenen Variablennamen aus. Ändern Sie jeweils nur eine Sache, beobachten Sie die Reaktion und beheben Sie den Fehler. Lassen Sie Ihrer Neugier freien Lauf: Fehlermeldungen helfen Ihnen, Ihren Code zu untersuchen!

Wichtig: Starten Sie nach einem Compilerfehler nicht einfach weiter mit `java Main`. Eine ältere `Main.class` kann noch vorhanden sein; dann würden Sie die alte Programmfassung ausführen. Korrigieren und speichern Sie zuerst den Quelltext und kompilieren Sie erfolgreich neu.

2.7 Den Quelltext aufbewahren und weiterbearbeiten

Speichern Sie Ihr Programm und schließen Sie die Datei im Editor. Öffnen Sie danach genau diese `Main.java` wieder. Ändern Sie eine Begrüßung, speichern Sie und führen Sie die neue Fassung aus. Können Sie den Ablauf schon ohne Anleitung erklären? Lassen Sie diesmal Ihre Coding-Partner:in erklären und ergänzen Sie gemeinsam, was noch fehlt.

Bewahren Sie die `.java`-Datei auf: Sie enthält Ihren bearbeitbaren Quelltext. Die `.class`-Datei lässt sich daraus erneut erzeugen. Anders als beim bisherigen Experimentieren in der JShell haben Sie nun eine Datei, an der Sie später weiterarbeiten können. Auch in Aufgabe 4 greifen Sie darauf zurück.

2.8 Bonus: Begrüßen Sie namentlich!

Sie möchten weiterforschen? Erweitern Sie Ihr Programm so, dass eine Person ihren Namen im Terminal eingeben kann und anschließend persönlich begrüßt wird. Beispielsweise soll die Eingabe `Brian Kernighan` zur Ausgabe `Hallo Brian Kernighan!` führen.

Dies ist ein freiwilliger Vorgriff. Mit einem `Scanner` können Sie eine Eingabe lesen. Ergänzen Sie ganz oben in Ihrer Datei, vor der Klasse:

```
import java.util.Scanner;
```

Innerhalb von `main` helfen Ihnen diese Anweisungen:

```
Scanner eingabe = new Scanner(System.in);
System.out.println("Wie heißen Sie?");
String name = eingabe.nextLine();
```

Ergänzen Sie selbst die Begrüßung. Wenn das gestartete Programm nach dem Namen fragt, geben Sie diesen im Terminal ein und bestätigen ihn mit der Eingabetaste. `nextLine()` liest die ganze Zeile, also auch Namen mit Leerzeichen. Wie könnten Sie die Begrüßung noch persönlicher gestalten? Probieren Sie es aus!

Ergänzende Lernquellen finden Sie in Moodle und in der folgenden freiwilligen Aufgabe.

3 Crashkurs

Strecklernziele

- ☐ Sie verschaffen sich einen kurzen Überblick über die Programmiersprache Java.
- ☐ Sie erkennen Lücken in Ihrem Vorwissen und finden Wege, diese zu schließen.
- ☐ Sie beschäftigen sich mit ergänzenden und alternativen Lern- und Informationsquellen.

Diese Aufgabe ist freiwillig. Mit ihr verfolgen Sie *Strecklernziele*, die auch als *Stretch-* oder *erweiterte Lernziele* bezeichnet werden. Diese erweiterten Lernziele gehen über die normalen Lernziele hinaus. Mit dem Erreichen dieser Lernziele können Sie z. B. Ihr Wissen über ein Thema vertiefen und erweitern oder es besser verstehen und lernen.

3.1 Learn Java in One Video - 15-minute Crash Course

Sehen Sie sich das Video Learn Java in One Video - 15-minute Crash Course an. Es gibt Ihnen einen schnellen Überblick über die Programmiersprache Java.

Tipp: Das Video ist auf Englisch. Pausieren Sie bei Bedarf oder reduzieren Sie die Wiedergabegeschwindigkeit, beispielsweise auf das 0,75-Fache. Aktivieren Sie Untertitel, sofern diese verfügbar sind.

Es geht nicht darum, alles zu verstehen, sondern einen ersten Eindruck von der Sprache Java zu bekommen. Achten Sie darauf, welche Begriffe Sie aus Kapitel 1 oder Ihren bisherigen Experimenten wiedererkennen und welche Sie später genauer kennenlernen möchten.

3.1.1 Beobachtungsauftrag

Damit Sie beim Anschauen einen Anhaltspunkt haben, erhalten Sie einen Beobachtungsauftrag:

- Notieren Sie sich beim Schauen drei Begriffe oder Konzepte, die Ihnen neu oder besonders interessant erscheinen, zum Beispiel *Variable*, *Schleife* oder *Methode*.

- Bewerten Sie an dieser Stelle nicht, ob Sie sie verstehen. Halten Sie sie als Anknüpfungspunkte für später fest.

3.2 Selbsteinschätzung

Wenn Ihnen vieles noch unbekannt vorkommt, ist das beim Einstieg völlig in Ordnung. Ein schneller Überblick über viele neue Konzepte ist kein Maßstab dafür, was Sie bereits können sollten. Wählen Sie einen Begriff aus, der Sie neugierig gemacht hat. Finden Sie ihn in Ihrem Buch wieder? Können Sie ein kleines Beispiel dazu ausprobieren?

Überlegen Sie, welche Unterstützung Ihnen dabei hilft: das Buch, eine andere Lernquelle, Ihre Coding-Partner:in oder Ihre Lernbegleiter:innen. Halten Sie auch fest, welche Frage noch offenbleibt.

3.3 Lern- und Informationsquellen

Im Moodle-Kurs finden Sie ergänzende und alternative Lern- und Informationsquellen. Insbesondere finden Sie dort eine Liste von Büchern, die Ihnen helfen können, die Grundlagen des Informatikstudiums zu verstehen: *Vorkurs Informatik für Dummies*, *Programmieren lernen für Dummies* und *Java für Dummies*. Auch wenn die Titel nicht gerade schmeichelhaft klingen, sind die Bücher sehr gut und verständlich geschrieben.

Schauen Sie sich die Liste an und überlegen Sie, ob Sie eines der Bücher lesen möchten. Was könnte Ihnen dadurch möglich werden?

4 Entdeckungsreise

Strecklernziele

- ☐ Sie befassen sich mit grundlegenden Begriffen und Konzepten der Programmierung.
- ☐ Sie wenden grundlegende Konzepte der Programmierung in Ihrem eigenen Java-Code an.

Auch diese Entdeckungsreise ist freiwillig. Kapitel 1 des Buches und das Video zu Aufgabe 3 stellen Ihnen wichtige Konzepte und Begriffe der Java-Programmierung vor. Dazu gehören Variablen, Datentypen, Strings, Schleifen und Bedingungen. Wählen Sie das Buch oder das Video als Ausgangspunkt für Ihre Entdeckungsreise; Sie müssen das Video dafür nicht angeschaut haben.

Vieles davon ist so grundlegend, dass es sich auf andere Programmiersprachen wie Python, C++ oder JavaScript übertragen lässt. Während des Studiums und im späteren Berufsleben werden Sie mit verschiedenen Programmiersprachen in Kontakt kommen. Diese Grundlagen gehören also zu Ihrem Handwerkszeug als Programmierer:in.

4.1 Reflektieren Sie Grundlagen der Programmierung

Diskutieren Sie im Coding-Team oder mit anderen Kommiliton:innen, welche grundlegenden Konzepte der Programmierung Ihnen in Kapitel 1 oder im Video begegnet sind. Die folgenden Fragen können dabei helfen:

- Welche Begriffe und Konzepte sind Ihnen in Ihrer gewählten Quelle begegnet?
- Welche Begriffe und Konzepte sind Ihnen bereits bekannt?
- Welche Begriffe und Konzepte sind Ihnen neu?
- Welche Begriffe und Konzepte erscheinen Ihnen besonders wichtig?
- Wenn Sie aus der Sicht von Benutzer:innen darüber nachdenken, was Programme typischerweise tun, was könnten Anwendungsfälle für diese Begriffe und Konzepte sein?

4.2 Erweitern Sie Ihr Java-Programm

Öffnen Sie Ihren gespeicherten Quelltext und erweitern Sie Ihr Java-Programm aus Aufgabe 2 so, dass es mindestens zwei der Konzepte aus Kapitel 1 oder dem Video verwendet. Probieren Sie es aus! Wenn Sie sich unsicher sind, fragen Sie Ihre Kommiliton:innen oder Ihre Lernbegleiter:innen um Hilfe. Hier sind einige Vorschläge:

Schleife Verwenden Sie eine `while`-Schleife, um die Begrüßung mehrmals auszugeben. Beispiele finden Sie auf den Seiten 13 und 14 des Buches. Alternativ können Sie eine `for`-Schleife ausprobieren. Achten Sie darauf, dass Ihre Schleife nach der gewünschten Anzahl von Wiederholungen endet.

Alter statt Name Speichern Sie zunächst ein Alter in einer Variable, etwa `int alter = 20;`, und geben Sie eine passende Nachricht aus. Eine Eingabe über die Tastatur ist dafür nicht erforderlich.

`if`-Bedingung Verwenden Sie eine `if`-Bedingung, um eine Ausgabe zu erzeugen, die sich je nach Alter der Person unterscheidet, aber dennoch schmeichelhaft ist. Beispiele zu `if` finden Sie auf Seite 15 des Buches.

Die Bonusaufgabe zur Namenseingabe müssen Sie hierfür nicht bearbeitet haben. Wenn Sie Eingaben bereits ausprobiert haben, können Sie Ihr Programm zusätzlich um eine Alterseingabe erweitern. Denken Sie auch hier daran, nach Änderungen zu speichern, zu kompilieren und auszuführen.